Les tables de hachage

I) Définition d'une table en général

Une table est une structure de données telle que l'accès à un élément est effectué à partir de sa clé. La clé permet, en général, d'identifier un élément de manière unique. Connaissant la clé, on pourra retrouver l'élément associé.

L'allocation d'espace pour une table est souvent contiguë, et s'effectue en début de traitement.

	Clé	Informations
0		
1		
2	Dupond	*Informations spécifiques (prénom, adresse, téléphone, adresse électronique ...)*
3		
....		

Lorsque les informations sont trop nombreuses pour résider en mémoire centrale, on peut les stocker en mémoire secondaire. On constitue alors une table d'index en associant à chaque clé un numéro d'enregistrement, celui de l'enregistrement contenant les informations relatives à cette clé.

Table des index (en mémoire centrale pendant l'exécution du programme) :

	Clé	Numéro d'enregistrement
0		
1		
2	Dupond	*25*
3		
....		

Fichier de données (en mémoire secondaire y compris pendant l'exécution du programme) :

N° de l'enregistrement	Prénom	Adresse	Téléphone	E-mail
0				
1				
....				
25	Jacques	*50 av. de Bordeaux*	*05.65.77.10.80*	*dupond@iut-rodez.fr*
....				

La recherche des informations à partir de la clé peut se faire séquentiellement, ou bien si les clés sont ordonnées elle peut se faire par dichotomie.

Table à adressage direct :

Si la clé est un nombre entier, le plus simple est de l'utiliser en tant qu'indice du tableau. Ainsi l'information qui a pour clé 54 se trouvera à l'indice 54 du tableau. Cependant cette représentation ne convient que si l'ensemble des valeurs possibles pour les clés est de la forme [0; m] et m n'est pas trop grand.

Exemple : si la clé est le numéro du jour dans l'année, ou le numéro du salarié dans une entreprise à effectif réduit.

Contre-exemple : si la clé est le numéro INSEE g aa mm dd ccc ooo donc 13 chiffres

II) Les tables de hachage

Ces tables sont aussi nommées tables d'adressage dispersé. Le terme table de hachage est la traduction de ***hash table*** (il s'agit de hacher ou de découper la clé pour n'en conserver qu'une partie utile pour effectuer les recherches, les ajouts et suppressions des éléments).

Cette structure convient bien si le nombre de clés effectivement stockées est petit comparé au nombre potentiel de clés (exemple : les numéros INSEE). La clé n'est pas utilisée comme indice du tableau, mais cet indice est calculé à partir de la clé, ceci grâce à une **fonction de hachage**, appelée aussi **fonction d'adressage :**

h : Clé ---> Place

où *Place* est l'ensemble des positions possibles pour un élément de la table.

Le plus souvent les tables de hachage sont représentées par des tableaux et le rôle de la fonction de *h* est de retourner une valeur parmi les indices du tableau. Une fonction d'adressage "idéale" renvoie une place différente pour chacune des clés des éléments.

Exemple :

Prenons une table qui possède 11 places, dans laquelle on désire insérer 7 éléments dont les clés sont : Louise, Germaine, Paul, Maud, Pierre, Jacques et Monique. Supposons que la fonction de hachage renvoie respectivement : 6, 2, 8, 3, 1, 7 et 10. On obtient la table suivante :

	Clé	Informations
0		
1	Pierre	.....
2	Germaine	.....
3	Maud	.....
4		
5		
6	Louise	.....
7	Jacques	.....
8	Paul	.....
9		
10	Monique	.....

Les éléments dans les tables de hachage ne sont pas ordonnés et le coût de la recherche d'un élément est constant et ne dépend pas du nombre d'éléments de la table (différence avec les tables précédentes)

(la recherche se limite à calculer la valeur de la fonction de hachage et à déterminer si la valeur est présente ou pas).

Cette structure de données est très souvent utilisée :

- ✓ par les compilateurs pour implémenter les tables de symboles
- ✓ par les correcteurs orthographiques des traitements de texte.
- ✓ pour un annuaire
- ✓ ...

Elle est très performante à condition de bien choisir la fonction de hachage, la taille de la table et la façon de traiter les collisions.

2.1) Choix de la fonction d'adressage

Très souvent, la fonction d'adressage (ou de hachage) renvoie une même place pour des valeurs différentes. On dit alors qu'il y a **collision**.

Une bonne fonction d'adressage doit à la fois répartir les clés le plus uniformément sur l'ensemble des places de la table et être simple et rapide à calculer (la rapidité est importante, car a priori la fonction serait invoquée très souvent).

Toute fonction de hachage doit d'abord passer à une représentation entière de la clé (sauf si la clé est déjà sous cette forme) et donner un indice dans la table, en passant donc par le calcul d'un modulo *m*, si *m* est la taille de la table.

En général, la clé est une chaîne de caractères. Le calcul peut s'effectuer à partir d'une partie seulement des caractères de la chaîne. Voici quelques méthodes couramment utilisées :

- ✓ prendre 4 caractères au centre de la chaîne
- ✓ prendre les 3 premiers et les 3 derniers caractères (en écartant les blancs)
- ✓ regrouper les caractères par bloc de 4 pour calculer une valeur entière
- ✓ calculer des suites de la formes h_0, $h_i = \alpha\, h_{i-1} + c_i$ sur la totalité de la chaîne, ou sur la portion jugée la plus significative

C'est cette dernière manière qui a été employée dans l'exemple avec les prénoms.

```
int fonctionAdressage(String s) {
    int h = 0;
    for (int i = 0; i < s.length(); i++) {
        h = 32 * h + s.charAt(i);      // ou  h = (h < 5) + s.charAt(i);
    }
    return Math.abs(h) % TAILLE;       // TAILLE est la taille de la table
}
```

On prend la valeur absolue de *h,* car à la sortie de la boucle la valeur de *h* peut être négative à cause d'un débordement.

Il est préférable que le coefficient de la multiplication soit une puissance de 2. En effet, de manière interne, de telles multiplications peuvent se faire très facilement par des décalages de bit.

En Java, une autre stratégie est utilisée. Un masque binaire est appliqué sur le nombre obtenu par le calcul et la partie significative du code de hachage est celle portée par les bits de poids faible. La formule de calcul pour les chaînes est donc : `h = 31 * h + s.charAt(i);`
`31` étant un nombre premier, la multiplication ne se résume pas à un décalage des bits de poids de faibles.

Il n'existe pas de fonction d'adressage universelle. En fait, elle dépend de l'ensemble des clés utilisé. Une fonction efficace pour des noms et des prénoms ne le sera pas nécessairement pour les mots réservés d'un langage de programmation, ou pour des codes de sécurité sociale.

Si l'ensemble des clés est connu, il est possible de chercher une fonction sans collision. Il existe des générateurs de fonctions d'adressage qui, à partir d'un ensemble de clés connu, fournissent des fonctions d'adressage sans collision. Si par contre l'ensemble des clés n'est pas connu, on choisira une fonction simple à calculer.

Le choix de la taille *m* de la table est également stratégique. Il faudra réserver une taille supérieure au nombre d'éléments prévus, mais sans exagérer pour ne pas gaspiller de l'espace. Le taux de remplissage de la table, c'est-à-dire la valeur *n / m* où *n* est le nombre effectif d'éléments présents, et *m* la taille de la table, a une influence sur la qualité de la fonction d'adressage. Plus ce taux sera faible, plus le risque de collision sera réduit, mais au prix d'une perte de place mémoire.

Enfin, pour éviter une accumulation de collisions pour les clés dont la représentation entière serait un multiple de *m*, la taille de la table sera un nombre premier. Par contre, si l'on souhaite privilégier la rapidité de calcul, on choisira une puissance de 2.

2.2 Résolution des collisions

Il n'est pas toujours possible d'éviter les collisions, et il faut bien sûr que l'insertion dans la table soit possible même si la clé de l'élément à ajouter entre en collision avec celle d'un élément déjà présent dans la table. Il existe 2 stratégies pour résoudre les collisions : utiliser des fonctions secondaires ou chaîner les collisions de manière externe ou interne.

a) Utilisation de fonctions secondaires

Cette méthode consiste à appliquer, en cas de collision, une seconde fonction d'adressage pour obtenir une nouvelle place dans la table. Si cette nouvelle place provoque à nouveau une collision, on applique une troisième fonction d'adressage et ainsi de suite jusqu'à ce qu'il n'y ait plus de collision ou que le nombre de fonctions secondaires ait été épuisé.

Plus formellement :

$h_i(c) = (h(c) + f(i)) \bmod m$, avec $f(0) = 0$

hi est la fonction secondaire de rang i
h est la fonction principale

Le choix de ses fonctions secondaires est délicat. Le coût d'évaluation de ses fonctions doit rester faible. D'autre part, elles doivent disperser les éléments le plus uniformément possible sur toutes les places disponibles de la table.

Plusieurs méthodes : linéaire, quadratique et adressage double ...

1) méthode linéaire $f(i) = i$

Cette méthode revient à chercher séquentiellement la première place libre à partir de la place sur laquelle il y a collision.
Avantage : facile à implémenter, calcul rapide
Inconvénient : création de longues suites de cases occupées dans la table ce qui augmente le temps de recherche moyen.

2) méthode quadratique $f(i) = a\,i^2 + b\,i$

La fonction est un polynôme du second degré.
Pour éviter les collisions, il est préférable que a et b soient premiers avec m.
Si $i = 1$, on aura $a + b$, si $i = 2$: $4\,a + 2\,b$ si $i = 3$: $8\,a + 3\,b$

Avantage : reste facile à implémenter, et relativement efficace
beaucoup moins de longues suites de cases occupées

3) adressage double

Cette méthode consiste à disposer d'une deuxième fonction d'adressage h'. On obtient alors
$h_i(c) = (h(c) + i\,h'(c)) \bmod m$

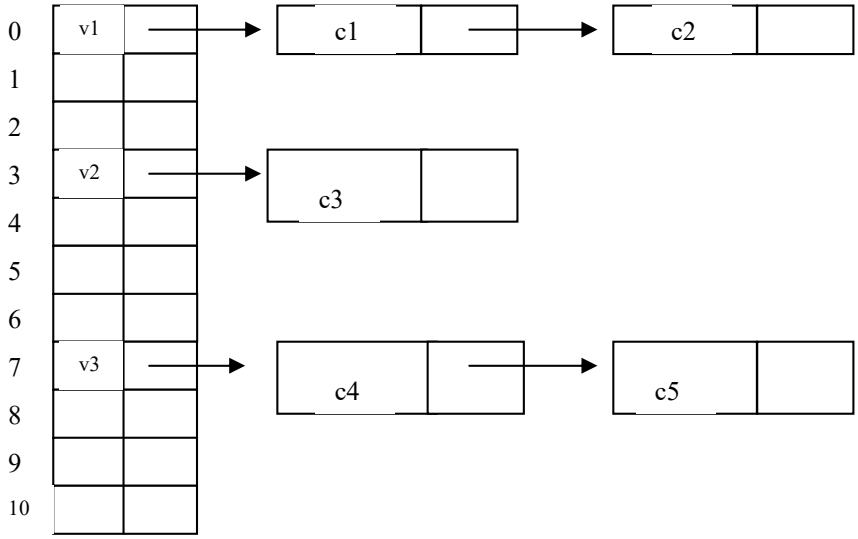
Cette méthode évite les accumulations et contrairement aux deux techniques précédentes produit pour 2 clés différentes en collision à cause du calcul de la première fonction de hachage, deux nouvelles places distinctes. Une fonction h' recommendée est : $h'(c) = 1 + (c \bmod (m - 2))$ avec m et $m - 2$ premiers.

b) Chaînage externe des collisions

Avec cette méthode, on ne cherche pas dans la table une nouvelle place pour les éléments dont les clés entrent en collision. Ceux-ci sont rangés dans des structures dynamiques en dehors de la table. Ces structures peuvent être des listes ou des arbres. Comme le nombre de collisions est en général faible, une liste suffit.

Le coût de cette méthode est celui de la fonction d'adressage primaire, plus celui de la recherche, de l'ajout ou suppression de l'élément dans la structure de données liste (ou arbre).

Cette méthode a l'avantage d'être très simple à mettre en oeuvre et permet une gestion efficace de la mémoire (le seul inconvénient est si la fonction d'adressage primaire provoquait un nombre de collisions anormal sur une même place de la table).



c) Chaînage interne des collisions

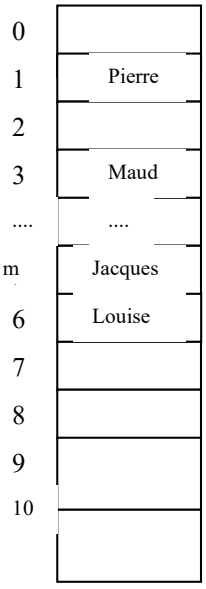
Le chaînage des collisions se fait dans la table elle-même. On peut définir dans celle-ci une zone d'adressage primaire et une zone d'adressage secondaire pour stocker les informations entrées en collisions. Le choix de la taille des 2 zones est la principale difficulté de cette méthode.

Exemple

Jacques est entré en collision avec Maud. Jacques est donc stocké dans la zone secondaire, au tout début de la zone de secondaire, s'il s'agit de la première collision qui se produit. On associe à Maud un lien vers l'emplacement de la table contenant Jacques (l'indice m).

Supposons qu'ensuite on souhaite stocker Louise, et qu'elle entre en collision avec Maud. Louise sera stockée en mémoire secondaire. On associe à Jacques un lien vers l'emplacement occupé par Louise.

Les éléments en collision sont donc chaînés les uns aux autres.



Il existe aussi une variante avec une seule zone. L'inconvénient est qu'il peut y avoir des collisions secondaires (qui pourraient être évitées).

d) Comparaisons des différentes méthodes

Lorsque le taux de remplissage est relativement faible, le nombre de collisions primaires est très faible et toutes les méthodes sont équivalentes.

Par contre, lorsque ce taux est important, et que donc le nombre de collisions augmente, le chaînage dans les structures externes est préférable.

En effet, l'adressage ouvert présente des inconvénients. D'une part, suite à une collision, il attribue une place à un élément qui peut très bien être celle du prochain élément à insérer, provoquant ainsi une nouvelle collision. D'autre part, les opérations de suppression d'un élément sont difficiles à mettre en oeuvre. Elles imposent de mémoriser le fait qu'il y a eu collision (sinon ce n'est pas le bon élément qui sera supprimé).

III) Les tables associatives en Java

Depuis Java 5, les tables associatives sont des classes génériques, au même titre que les collections. Elles sont dotées de deux paramètres : celui des clés (noté K) et celui des valeurs (noté V).

L'implémentation a bien sûr été optimisée. Deux types d'organisation sont proposée, via deux classes différentes :

- table de hachage : ***HashMap***. La clé est utilisée pour ranger les informations, plus précisément c'est le code de hachage des objets formant la clé qui est employé. Les collisions sont gérées, de manière interne, avec une liste chaînée.
- arbre binaire : ***TreeMap***. La clé est utilisée pour ranger les informations, plus précisément c'est la relation d'ordre induite par la méthode *compareTo* (ou un comparateur fixé à la construction) qui détermine la place des éléments. Il faut donc que la relation d'ordre ait un sens pour les clés.

Ces deux classes implémentent une interface nommée ***Map***.

3.1) Les principales méthodes de la classe *Map*.

L'interface ***Map<K, V>*** est paramétrée par le type ***K*** de la clé et le type ***V*** des valeurs.
Elle contient des méthodes pour :

- ✓ ajouter des associations <clé, valeur>
- ✓ supprimer des associations <clé, valeur>
- ✓ examiner le contenu d'une ***Map*** : tester l'existence d'une association à partir de la clé ou de la valeur, obtenir la valeur connaissant la clé, connaître la taille de la ***Map*** ...
- ✓ fournir des vues de la ***Map*** sous la forme de collections : des ensembles, ou d'autres collections

Une description plus détaillée de l'interface ***Map<K, V>*** est présente en annexe.

3.2) Les principales méthodes de la classe *HashMap*.

Cette classe implémente l'interface ***Map***. Elle possède plusieurs constructeurs. Celui qui n'a pas d'argument crée une table vide, comme par exemple :

```
HashMap<String, String> annuaire = new HashMap<>();
```

Dans les exemples de code ci-dessous, nous utiliserons comme exemple un annuaire téléphonique. Nous supposerons qu'il s'agit de l'annuaire interne de l'IUT : la clé sera le nom d'un service ou d'une personne (une chaîne de caractères) et le l'information le numéro de téléphone (une chaîne de caractères) .

Ajout d'information

On ajoute des éléments grâce à la méthode ***put*** :

```
annuaire.put("IUT", "05.65.77.10.80");
```

Le premier argument est la clé, le deuxième la valeur. Si la clé donnée en argument existe déjà dans la table, la valeur associée remplacera l'ancienne. En effet, une clé ne peut figurer qu'une seule fois dans la table. La méthode ***put*** renvoie comme résultat :

- ✓ *null*, si la clé n'existait pas déjà,
- ✓ la valeur associée à l'ancienne clé si un remplacement a été effectué.

Recherche d'information

On obtient la valeur associée à une clé donnée à l'aide de la méthode ***get***, laquelle renvoie *null* si la clé recherchée n'est pas présente.

```
String information;         // information récupérée dans la table
information = annuaire.get("IUT");
System.out.println("Le numéro de l'IUT est "
                   + (information == null ? "inconnu" : information));
information = annuaire.get("Université");
System.out.println("Le numéro de l'université est "
                   + (information == null ? "inconnu" : information));
```

La méthode ***containsKey*** permet de déterminer si une clé est présente.

```
if (annuaire.containsKey("IUT")) {
    System.out.println("Le numéro de l'IUT est présent dans l'annuaire.");
} else {
    System.out.println("Le numéro de l'IUT n'est pas présent dans l'annuaire.");
}

if (annuaire.containsKey("Université")) {
    System.out.println("Le numéro de l'Université est présent dans l'annuaire.");
} else {
    System.out.println("Le numéro de l'Université n'est pas présent.");
}
```

Suppression d'information

On peut supprimer un élément de la table en utilisant la méthode ***remove***. Celle-ci renvoie *null* si la clé à supprimer n'est pas présente dans la table, ou bien la valeur associée à la clé si la suppression a bien été effectuée.

```
information = annuaire.remove("Centre de documentation");
if (information != null) {
     System.out.println("Suppression du numéro " + information
                        + " pour le centre de documentation effectuée.");
}
information = annuaire.remove("Université");
if (information == null) {
    System.out.println("Suppression impossible du numéro de l'université.");
}
```

3.3) Parcours d'une table : notion de vue

En théorie, la classe ***HashMap*** ne dispose pas de la notion d'itérateur. Mais on peut malgré tout effectuer un parcours de la table, en la considérant comme un ensemble de paires. Chaque paire est un élément de type ***Map.Entry***, une interface définie dans l'interface ***Map*** (voir annexe pour une description plus détaillée), donc une association entre une clé et sa valeur.

Cette interface dispose des méthodes suivantes :

- ***getKey*** qui renvoie la clé de la paire
- ***getValue*** qui renvoie la valeur de la paire
- ***setValue*** avec en argument la nouvelle valeur de la paire. La méthode renvoie l'ancienne valeur de la paire.

La méthode ***entrySet*** renvoie, sous la forme d'un ensemble, une vue des paires de la table. Cet ensemble de paires peut être ensuite parcouru comme dans l'exemple ci-dessous.

```
// déclaration d'un ensemble initialisé avec les paires de la table
Set<Map.Entry<String, String>> entreeAnnuaire = annuaire.entrySet();
System.out.println("----------------------------------------\n"
                   + "Annuaire de l'IUT\n");

// parcours de toutes les paires de l'ensemble
for (Map.Entry<String, String> paire : entreeAnnuaire) {
     System.out.println(paire.getKey() + "  " + paire.getValue());
}
System.out.println("----------------------------------------");
```

L'ensemble renvoyé par la méthode ***entrySet*** n'est pas une copie des informations figurant dans la table. Il s'agit d'une vue. Si l'on réalise des modifications dans la table, elles seront effectives également dans la vue.

3.4) Autres vues

On peut obtenir deux autres types de vue à partir d'une table :

- l'ensemble des clés, à l'aide de la méthode ***keySet***

```
// déclaration d'un ensemble initialisé avec les clés de la table
Set<String> ensembleCle= annuaire.keySet();

// parcours de l'ensemble pour afficher toutes les clés
System.out.print("Les clés de l'annuaire sont : ");
for (String cle : ensembleCle) {
    System.out.print(cle + " -- ");
}
```

- la collection des valeurs à l'aide de la méthode ***values***

```
  // déclaration d'une collection initialisée avec les valeurs de la table
  Collection<String> ensembleValeurs= annuaire.values();

  // parcours de la collection pour afficher toutes les clés
  System.out.println("\nLes numéros présents dans l'annuaire sont : ");
  for (String valeur : ensembleValeurs) {
      System.out.print(valeur + " -- ");
  }
```

Le résultat de cette méthode est une collection, et pas un ensemble, car certaines valeurs peuvent apparaître plusieurs fois, contrairement à l'ensemble des clés.

3.5) Variantes de la classe *HashMap*

Il existe des classes légèrement différentes de ***HashMap***. Elles implémentent aussi l'interface ***Map***. Les principales sont les suivantes.

LinkedHashMap

Cette classe garantit l'ordre dans lequel ses itérateurs renvoient les éléments. En fait, il y a deux ordres possibles : éléments renvoyés selon leur ordre d'insertion, ou bien renvoyés selon l'ordre dans lequel ils ont été accédés (de l'accès le plus récent à l'accès le plus ancien).

WeakHashMap

Une ***Map*** classique conserve des références à tous les objets qu'elle contient. Cela signifie que si une clé n'est plus accessible que via l'association présente dans la ***Map***, elle ne peut être réclamée par le ramasse-miettes. En général, ce comportement convient.

Avec une ***WeakHashMap***, ce comportement est différent. Il permet d'optimiser l'occupation mémoire. Une paire clé-valeur sera retirée de la table si la clé n'est plus référencée hors de cette association.

IdentityHashMap

Une ***IdentityHashMap*** diffère d'une ***HashMap*** classique car deux clés sont considérées comme égales uniquement s'il s'agit d'un même objet. La comparaison des clés s'effectuent sur l'identité (= =) et non sur ***equals***.

Il existe également des sous-interfaces de ***Map*** : ***SortedMap*** et ***NavigableMap***.

SortedMap ajoute à son contrat une clause garantissant que ses itérateurs parcourront l'association dans l'ordre croissant des clés.

NavigableMap étend ***SortedMap,*** et a été ajoutée depuis Java 6 pour pallier des défauts de ***SortedMap***. Elle étend les possibilités et permet par exemple d'accéder plus facilement à un sous-ensemble de la ***Map***, sous-ensemble défini sous la forme d'un intervalle.

IV) Annexe

4.1) Extrait de la documentation de l'interface *Map*.

Interface Map<K,V>

- Type Parameters:
  `K` - the type of keys maintained by this map
  `V` - the type of mapped values

Nested Class Summary

Modifier and Type	Interface and Description
`static interface`	**`Map.Entry<K,V>`** A map entry (key-value pair).

Method Summary

Modifier and Type	Method and Description
`void`	**`clear`**`()` Removes all of the mappings from this map (optional operation).
`boolean`	**`containsKey`**`(Object key)` Returns `true` if this map contains a mapping for the specified key.
`boolean`	**`containsValue`**`(Object value)` Returns `true` if this map maps one or more keys to the specified value.
`Set<Map.Entry<K,V>>`	**`entrySet`**`()` Returns a `Set` view of the mappings contained in this map.
`boolean`	**`equals`**`(Object o)` Compares the specified object with this map for equality.
`V`	**`get`**`(Object key)` Returns the value to which the specified key is mapped, or `null` if this map contains no mapping for the key.
`int`	**`hashCode`**`()` Returns the hash code value for this map.
`boolean`	**`isEmpty`**`()` Returns `true` if this map contains no key-value mappings.
`Set<K>`	**`keySet`**`()` Returns a `Set` view of the keys contained in this map.
`V`	**`put`**`(K key, V value)` Associates the specified value with the specified key in this map (optional operation).
`void`	**`putAll`**`(Map<? extends K,? extends V> m)` Copies all of the mappings from the specified map to this map (optional operation).
`V`	**`remove`**`(Object key)` Removes the mapping for a key from this map if it is present (optional operation).
`int`	**`size`**`()` Returns the number of key-value mappings in this map.
`Collection<V>`	**`values`**`()` Returns a `Collection` view of the values contained in this map.

4.2) Extrait de la documentation de l'interface *Map.Entry*.

Interface Map.Entry<K,V>

```
public static interface Map.Entry<K,V>
```

A map entry (key-value pair).

The `Map.entrySet` method returns a collection-view of the map, whose elements are of this class. The *only* way to obtain a reference to a map entry is from the iterator of this collection-view. These `Map.Entry` objects are valid *only* for the duration of the iteration; more formally, the behavior of a map entry is undefined if the backing map has been modified after the entry was returned by the iterator, except through the `setValue` operation on the map entry.

Modifier and Type	Method and Description
boolean	**equals**(Object o) Compares the specified object with this entry for equality.
K	**getKey**() Returns the key corresponding to this entry.
V	**getValue**() Returns the value corresponding to this entry.
int	**hashCode**() Returns the hash code value for this map entry.
V	**setValue**(V value) Replaces the value corresponding to this entry with the specified value (optional operation).

4.3) Exemple d'utilisation

```
/*
 * Utilisation de la classe HashMap
 * ExempleHashMap.java                                    10/22
 */
package collections;
import java.util.Collection;
import java.util.HashMap;
import java.util.Map;
import java.util.Set;

/**
 * Exemple d'utilisation de la classe HashMap
 * @author INFO2
 * @version 1.0
 */
public class ExempleHashMap {

    /**
     * Programme pour illustrer l'utilisation de HashMap
     * @param  args argument non utilisé
     */
    public static void main(String[] args) {

        /*
```

```
         *  création d'une table vide pour représenter l'annuaire de l'IUT :
         *  la clé et l'information associée sont des chaînes de caractères
         *  La clé sera le nom d'un service, ou d'une personne et l'information
         *  un numéro de téléphone (représenté avec une chaîne de caractères).
         */
        HashMap<String, String> annuaire = new HashMap<>();

        // on ajoute des informations dans la table, des couples (clé, valeur)
        annuaire.put("IUT", "05.65.77.10.80");
        annuaire.put("Département Informatique", "05.65.77.15.62");
        annuaire.put("Service vie étudiante", "05.65.77.15.65");
        annuaire.put("Fax IUT", "05.65.77.10.81");
        annuaire.put("Centre de documentation", "05.65.77.15.61");

        // recherche d'informations
        String information;         // information récupérée dans la table
        information = annuaire.get("IUT");
        System.out.println("Le numéro de l'IUT est "
                           + (information == null ? "inconnu" : information));
        information = annuaire.get("Université");
        System.out.println("Le numéro de l'université est "
                           + (information == null ? "inconnu" : information));

        if (annuaire.containsKey("IUT")) {
            System.out.println("Le numéro de l'IUT est présent dans l'annuaire.");
        } else {
            System.out.println("Le numéro de l'IUT n'est pas présent dans l'annuaire.");
        }

        if (annuaire.containsKey("Université")) {
           System.out.println("Le numéro de l'Université est présent dans l'annuaire.");
        } else {
            System.out.println("Le numéro de l'Université n'est pas présent.");
        }

        // suppression d'information
        information = annuaire.remove("Centre de documentation");
        if (information != null) {
            System.out.println("Suppression du numéro " + information
                               + " pour le centre de documentation effectuée.");
        }
        information = annuaire.remove("Université");
        if (information == null) {
            System.out.println("Suppression impossible du numéro de l'université.");
        }


        // Parcours 1 : parcours de l'annuaire complet
        // déclaration d'un ensemble initialisé avec les paires de la table
        Set<Map.Entry<String, String>> entreeAnnuaire = annuaire.entrySet();
        System.out.println("----------------------------------------\n"
                           + "Annuaire de l'IUT\n");

        // parcours de toutes les paires de l'ensemble (chacune est de type Map.Entry)
        for (Map.Entry<String, String> paire : entreeAnnuaire) {
            System.out.println(paire.getKey() + "  " + paire.getValue());
        }
        System.out.println("----------------------------------------");


        // Parcours 2 : parcours des clés uniquement
        // déclaration d'un ensemble initialisé avec les clés de la table
        Set<String> ensembleCle= annuaire.keySet();

        // parcours de l'ensemble pour afficher toutes les clés
        System.out.print("Les clés de l'annuaire sont : ");
        for (String cle : ensembleCle) {
```

```
            System.out.print(cle + " -- ");
        }


        // Parcours 3 : parcours des valeurs uniquement
        // déclaration d'une collection initialisée avec les valeurs de la table
        Collection<String> ensembleValeurs= annuaire.values();

        // parcours de la collection pour afficher toutes les clés
        System.out.println("\nLes numéros présents dans l'annuaire sont : ");
        for (String valeur : ensembleValeurs) {
            System.out.print(valeur + " -- ");
        }


        // test des accès sur les vues
        System.out.println("\n\nTest corrélation vue et map\n");
        annuaire.put("Concierge", "05.65.77.15.66");

        // parcours de toutes les paires de l'ensemble
        for (Map.Entry<String, String> paire : entreeAnnuaire) {
            System.out.println(paire.getKey() + "  " + paire.getValue());
        }
         annuaire.remove("Concierge");
        if (entreeAnnuaire.contains("Concierge")) {
            System.out.println("Problème");
        }

        if (annuaire.containsKey("Concierge")) {
            System.out.println("Problème.");
        }
        entreeAnnuaire.remove("IUT");
        if (entreeAnnuaire.contains("IUT")) {
            System.out.println("Problème");
        }
        if (annuaire.containsKey("IUT")) {
            System.out.println("Problème.");
        }
        System.out.println("----------------------------------------");
    }
}
```

Exemple d'exécution du programme

```
Le numéro de l'IUT est 05.65.77.10.80
Le numéro de l'université est inconnu
Le numéro de l'IUT est présent dans l'annuaire.
Le numéro de l'Université n'est pas présent.
Suppression du numéro 05.65.77.15.61 pour le centre de documentation effectuée.
Suppression impossible du numéro de l'université.
----------------------------------------
Annuaire de l'IUT

Service vie étudiante  05.65.77.15.65
Fax IUT  05.65.77.10.81
IUT  05.65.77.10.80
Département Informatique  05.65.77.15.62
----------------------------------------
Les clés de l'annuaire sont : Service vie étudiante -- Fax IUT -- IUT -- Département
Informatique --
Les numéros présents dans l'annuaire sont :
05.65.77.15.65 -- 05.65.77.10.81 -- 05.65.77.10.80 -- 05.65.77.15.62 --
```